An Intro to Smart Contracts

Presented by:
Asante Ntiro

DUNIA
YETU

Agenda

- WTF are Smart Contracts?
- Virtual Machine Environments
- Ethereum Virtual Machine (EVM) a closer look
- Gas
- Smart Contract Use Cases
- Cool Resources for Further Learning
- Q&A

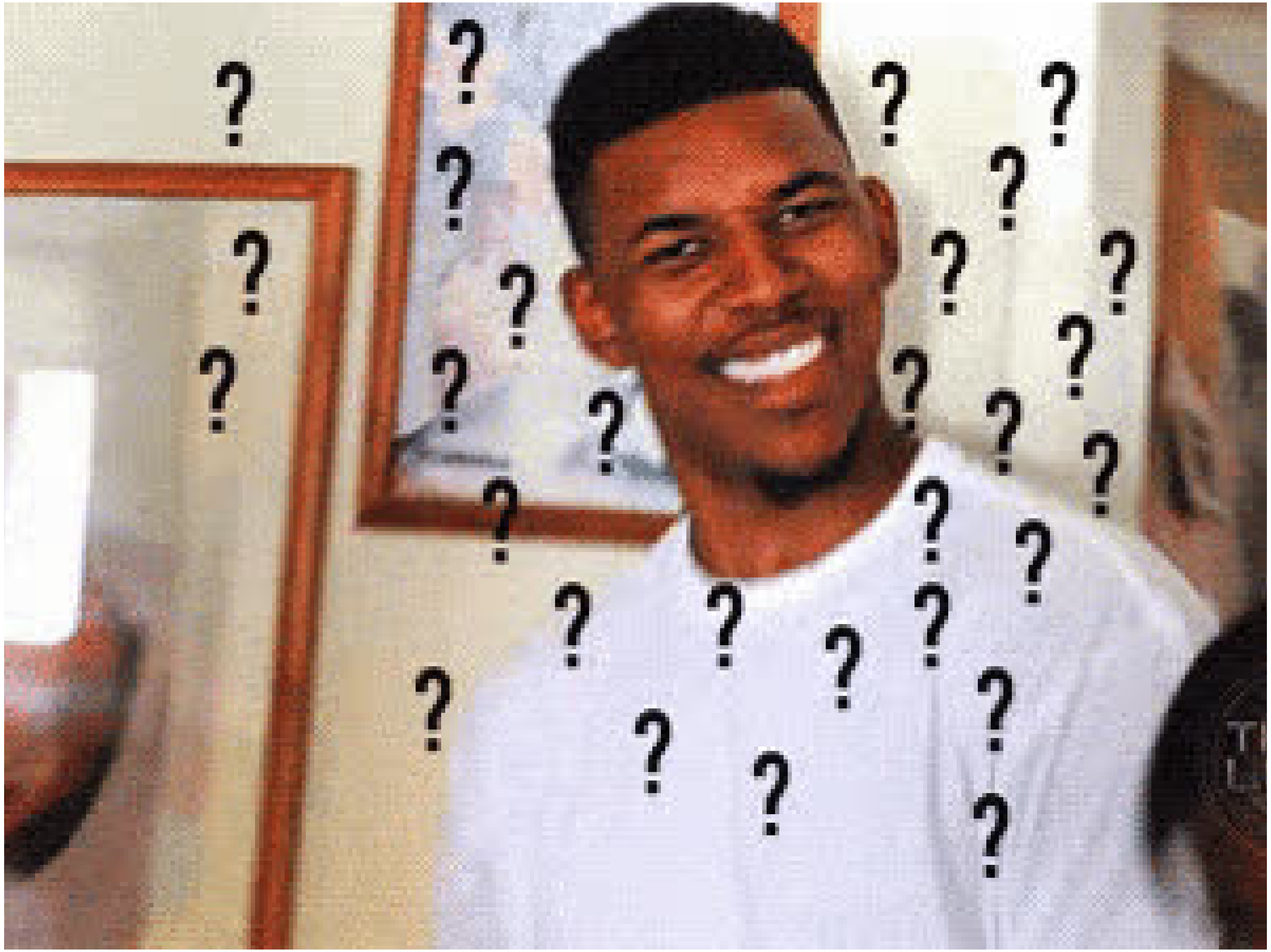
SECTION 1:

WTF ARE SMART CONTRACTS?

Smart contracts defined:

- **Self-executing digital agreements** with terms written directly in code.
- Automatically execute when predefined conditions are met.
- Run on decentralised blockchain networks.
- Execute exactly as programmed without downtime, censorship, or third-party interference.
- "Smart contracts combine protocols with user interfaces to formalise and secure relationships over computer networks." - Nick Szabo (1994)

Technology that enables: trustless, automated agreements



Basically…

Programs that run on **blockchains** 😅

Here’s some historical context!

1994

Nick Szabo first coined the term "smart contracts"

2009

Bitcoin introduced limited scripting capabilities

2015

Ethereum launched with Turing-complete smart contract platform

2020-Present:

Explosion of alternative smart contract platforms

Smart Contracts vs. Traditional Contracts

Traditional Contracts	Smart Contracts
Paper-based or digital documents	Computer code
Enforcement through legal system	Self-enforcing through code
Human interpretation	Deterministic execution
Third-party intermediaries	Decentralized validation
Manual execution	Automated execution

Does that mean that smart contracts are just the **digital version** of **“paper” contracts**??

Vitalik doesn't think so!

Key Properties of Smart Contracts

- **Immutability**: Once deployed, code cannot be changed
- **Transparency**: Code is visible to all network participants
- **Trustlessness**: No need to trust counterparties or intermediaries
- **Deterministic**: Same inputs always produce the same outputs
- **Distributed**: Execution verified by multiple nodes

Smart Contract Architecture

- Code: Logic written in specialised languages (Solidity, Vyper, Soroban, MOVE... etc)
- State: Current data values stored on the blockchain
- ABI: Application Binary Interface that defines how to interact with the contract
- Bytecode: Low-level compiled code executed by the virtual machine

Lifecycle of a Smart Contract

1. **Development:** Writing and testing the contract code
2. **Compilation:** Converting high-level code to bytecode
3. **Deployment:** Publishing the contract to the blockchain
4. **Execution:** Functions called by users or other contracts
5. **Termination:** Self-destruct or permanent inactivity

SECTION 2:

VIRTUAL MACHINE ENVIRONMENTS…

What is a Blockchain Virtual Machine?

1. **Runtime environment for executing smart contracts**
2. **Abstraction layer between contract code and underlying blockchain**
3. **Sandboxed execution environment for security**
4. **State machine that transitions based on transactions**
5. **Consensus-based validation of execution**

Major Virtual Machine Environments

- **Ethereum Virtual Machine (EVM):** Ethereum's runtime environment
- **Bitcoin Script:** Bitcoin's limited scripting capabilities
- **Solana's Sealevel:** Parallel smart contract runtime
- **WebAssembly (WASM):** Used by Polkadot, NEAR, etc.
- **Cosmos SDK:** Framework for building application-specific blockchains
- **Move VM:** Aptos and Sui's resource-oriented VM

EVM (Ethereum Virtual Machine)

- **Architecture:** Stack-based, 256-bit register machine
- **Languages:** Solidity, Vyper
- **Features:** Turing-complete, account-based
- **Adoption:** Most widely used smart contract platform
- **Limitations:** Sequential execution, limited throughput

Programming Language Comparison

Platform	Primary Languages	Learning Curve	Developer Experience
EVM	Solidity, Vyper	Moderate	Extensive tooling
Bitcoin	Script	Steep	Limited tooling
Solana	Rust	Steep	Growing ecosystem
Polkadot	Rust (for WASM)	Steep	Modern tooling
Cosmos	Go	Moderate	Framework-focused

VM Trade-offs

Security vs. Flexibility:

More features = larger attack surface

- **Bitcoin Script:** limited but secure
- **EVM:** flexible but more vulnerable

Performance vs. Decentralization:

Higher throughput = more centralization

- **Solana:** high performance, higher hardware requirements
- **Ethereum:** more decentralized, lower throughput

SECTION 3:

EVM
A CLOSER LOOK 👀

EVM Architecture

- Stack-based virtual machine
- 256-bit word size (optimized for cryptography)
- Quasi-Turing complete (gas-limited)
- Deterministic execution
- Memory model: Stack, Memory, Storage
- Accounts: External (users) and Contract accounts

EVM Architecture

Stack:

- **Temporary values during execution (limited to 1024 elements)**

Memory:

- **Volatile linear memory (expanded as needed)**

Storage:

- **Persistent key-value store (expensive to use)**

EVM-Compatible Chains

- **Ethereum:** Original EVM blockchain
- **BNB Chain:** Binance's EVM-compatible chain
- **Base:** Ethereum scaling solution (L2)
- **Optimism/Arbitrum:** Layer 2 rollups for Ethereum

okay let's get to the fun stuff!

***rubs hands*

SECTION 3.2:

SOLIDITY BASICS… 🤖

LET'S OPEN UP

THE REMIX IDE

Contract Structure

- **Pragma directive**: Specifies compiler version
- **Contract declaration**: Similar to a class in OOP
- **State variables**: Persistent storage
- **Functions:** Code executed when called
    - Public, private, internal, external visibility
    - View, pure, payable modifiers
- **Events**: Logging mechanism
- **Modifiers**: Reusable code for function preconditions

SECTION 4:

GAS...

Gas?

What do you think would happen if we created an infinite loop in a smart contract?

If a smart contract had an unbounded or infinite loop it could:

If a smart contract had an unbounded or infinite loop it could:

deny service to other users!

Gas!

Gas is the mechanism Ethereum uses to limit the computational resources a transaction or smart contract can consume.

Every operation in the Ethereum Virtual Machine (EVM) has a gas cost.

Gas!

- Unit of computational work in the EVM
- Prevents infinite loops and DoS attacks
- Incentivizes efficient code writing
- Compensates validators for resources used
- Creates market dynamics for transaction inclusion

Economic Implications of Gas fees

- **Accessibility:** High fees exclude small transactions
- **Prioritization:** Creates market for transaction inclusion
- **Value capture:** Validators/miners revenue
- **Network security:** Fees contribute to security budget
- **Layer 2 solutions:** Batching transactions to reduce costs
- **MEV (Maximal Extractable Value):** Profit from transaction ordering

SECTION 5:

SMART CONTRACT USE CASES

DeFi (Decentralized Finance)

- **Automated Market Makers (AMMs)**
  - **Uniswap, Curve, PancakeSwap**
- **Lending Protocols**
  - **Aave, Compound, MakerDAO**
- **Stablecoins**
  - **DAI, USDC, FRAX**

NFTs and Digital Ownership

- Art and Collectibles
  - CryptoPunks, Bored Ape Yacht Club
- Domain Names
  - ENS (Ethereum Name Service)
- Membership Passes
  - Access tokens, event tickets

DAOs and Governance

- Protocol Governance
  - Uniswap, Compound
- Investment DAOs
  - MetaCartel, The LAO
- Service DAOs
  - Developer DAOs, Creator DAOs
- Social DAOs
  - Farcaster, Bankless

Supply Chain & Real-World Assets

- Product Provenance
- Tokenized Real Estate
- Agricultural Tracking
    - IBM Food Trust
- Luxury Goods Authentication

Enterprise Applications

- Document Verification
  - Certifications, credentials
- Intellectual Property
  - Rights management, royalty distribution
- Compliance and Auditing
  - Automated reporting
- Tokenized Securities
  - Regulated token offerings

Other cool stuff..

- Decentralized Science (DeSci)
  - Research funding, peer review
- Social Networks
  - Decentralized reputation systems
- Identity Solutions
  - Self-sovereign identity
- Public Goods Funding
  - Quadratic funding, impact certificates

Key Takeaways:

- Smart contracts are **self-executing** agreements with **code-defined terms**
- Multiple VM environments exist with different **trade-offs**
- The EVM is the **most widely** adopted smart contract platform
- Gas fees create **economic incentives** and **security measures**
- Use cases span finance, art, governance, **and beyond**
- The technology is still evolving **rapidly**

Resources for Further Learning

Documentation:

- Ethereum.org Developer Resources
- Solidity Documentation (latest version)
- Ethereum Study Group Wiki -> 🌟

Practice Platforms:

- Remix IDE
- CryptoZombies
- SpeedRunEthereum -> 🌟

DUNIA YETU
www.youtube.com/@DuniaYetu-TZ

find me on x:
@AsanteNtiro

Thank you..